

Name:

Punkte/Note:

Bevor Sie beginnen

- Franklyn muss gestartet und aktiv sein.
Es erfolgt keine Bewertung, wenn Franklyn beendet wird (z.B. durch das Sperren des Bildschirms).
- Das Angabelaufwerk finden Sie hier: `\\edufs\testSall`
- Kopieren Sie vom Angabelaufwerk das gesamte Verzeichnis „LF3“ auf ihr U: („home“) Laufwerk.
- Ändern Sie anschließend den Namen des gerade kopierten Verzeichnisses (am U: Laufwerk).
Aus LF3 wird `MustermannMax` (Ihr Name, Nachname zuerst und ohne Leerzeichen)
- In dem Verzeichnis finden Sie eine Schablone für die Aufgabe.
- VS2022 starten Sie, indem Sie auf die `sln` Datei (im Unterverzeichnis) klicken.

Bewertungshinweise

- Bewertet werden nur Quelldateien, die in der oben beschriebenen Verzeichnisstruktur (auf U:) liegen. Quelldateien am „Desktop“ oder im Ordner „Dokumente“ sind für die Bewertung nicht sichtbar und werden daher nicht berücksichtigt.
- Das Programm muss nicht nur funktionieren, es muss auch mit einem entsprechenden Stil geschrieben sein (der Stil wird auch bewertet).
Achten Sie auf Variablenbezeichnungen, Einrückungen, Leerzeilen, Lesbarkeit der Lösung, ...
- Der C# Compiler hilft Fehler zu finden. Quelldateien müssen für eine positive Bewertung fehlerfrei übersetzt werden können.
- Falsche Einrückungen werden wie C# Compilerfehler gewertet.
- Gleiche bzw. idente Lösungen unterschiedlicher Schüler werden nicht bewertet!

Aufgabe 1: Anagram

Ein **Anagramm** (im Volksmund: **Schüttelwort**) besteht aus denselben Buchstaben wie das Originalwort, allerdings sind diese Buchstaben an beliebigen Stellen angeordnet („durchgeschüttelt“). **Beispiel:** Voldemort => Tomrodlev

Schreiben Sie Methoden zur Behandlung von Anagrammen:

- `bool IsAnagram(string text, string compareWith)` [25Pkt.]
Dies Methode überprüft, ob die beiden übergebenen Texte Anagramme sind. Dabei gilt:
 - Groß/Kleinbuchstaben werden nicht unterschieden.
 - Es sind nur Buchstaben (`char.IsLetter`) in den Texten enthalten.
 - Die beiden Texte müssen gleich lang sein.
 - Die beiden Texte dürfen nicht gleich sein.
 - Jedes Zeichen aus dem ersten Text hat eine exakte Entsprechung im zweiten.
- `int[] GetAnagramConversion(string text, string compareWith)` [25Pkt.]
Dies Methode berechnet eine Konvertierungstabelle, wie vom ersten Text zum zweiten gelangt werden kann.
Beispiel: `Brei <-> Bier`. Die Konvertierung ist `0, 3, 2, 1`.
`B` kommt an die Stelle 0, `r` ist im zweiten Text an der Stelle 3, usw.
Die Methode liefert als Rückgabewert `null` wenn keine Anagramme übergeben wurden.
- `string ConvertAnagram(string text, int[] conversion)` [25Pkt.]
Mit dieser Methode wird von dem ersten Text und einer Konvertierungstabelle – z.b. berechnet durch die Methode `GetAnagramConversion` – der zweite Text berechnet.

In dem bereitgestellten Programm-Template sind `UnitTests` vorhanden. Implementieren Sie die Methoden so, dass **alle** Unittests erfolgreich ausgeführt werden können. Änderungen an den Unittests sind nicht erlaubt.

[25Pkt.] Diese drei Methoden sollen/müssen in einem Hauptprogramm aufgerufen werden. Lesen Sie zwei Texte ein (Mit einer Länge von 1 bis 30 Zeichen, wiederholen Sie bei Fehlern die Eingabe), prüfen Sie ob die beiden Texte Anagramme sind und geben Sie die Konvertierungstabelle aus. Orientieren Sie sich an der folgenden Beispielausgabe:

BildschirmAusgabe

```
*** Anagram Checker ***  
  
Please enter 1. word, Length:1-30):  
Invalid input, try again...  
Please enter 1. word, Length:1-30): Bier  
Please enter 2. word, Length:1-30): Brei  
Base: "B i e r"  
Idx:  "0 3 2 1"  
To:   "B r e i"  
Calc: "B r e i"
```

Hinweis: `To` ist der eingegebene Text, `Calc` der errechnete Text.

Oder

```
Please enter 1. word, Length:1-30): Bier  
Please enter 2. word, Length:1-30): Saft  
"Bier" and "Saft" are no anagrams!
```